

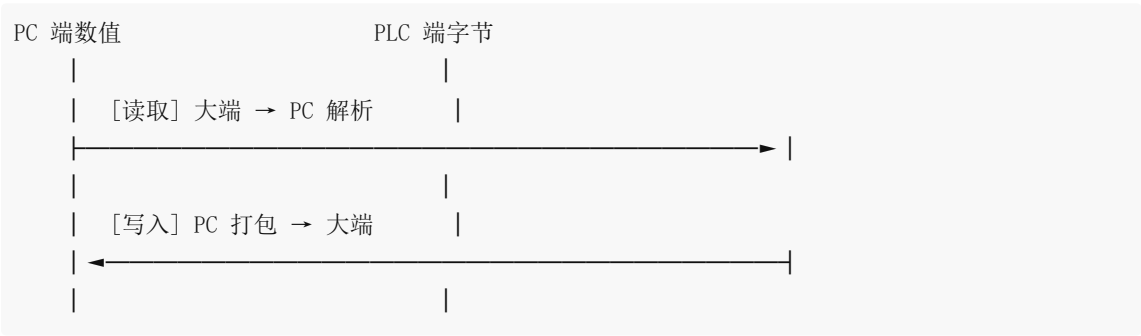
Qt + Snap7 读写 PLC 数据 —— 小白上手指南（项目 4）

本项目和项目 3 的区别

	项目 3	项目 4
功能	连接 + 读取	连接 + 读取 + 写入
新增控件	—	写入区域选择、值输入框、写入按钮
新增核心知识	—	大端字节序打包 （和读取时相反的转换）
自动回读验证	—	写入后立即读取同一地址，确认写入成功

写入和读取的关系

读取是把 PLC 的"大端字节"转成 PC 能理解的数值，写入正好反过来：



项目文件一览

stage4_write_data/	
├── CMakeLists.txt	
├── main.cpp	← 同项目3
├── mainwindow.h	← 新增写入控件 + packValue 声明
├── mainwindow.cpp	← 新增写入逻辑 + 数据打包函数（核心新增）
├── plc_worker.h	← 新增 writeArea 槽 + dataWritten 信号
├── plc_worker.cpp	← 新增 DBWrite/MBWrite/EBWrite/ABWrite 调用
├── snap7.h / snap7.cpp / snap7.dll / libsnap7_mingw.a	

核心概念：大端字节序打包 (packValue)

项目 3 教了怎么把 PLC 的大端字节"拆开"成数值。项目 4 教怎么把数值"装回去"（打包）。

举例：把 INT16 数值 -100 写入 PLC

PC 内存中的 -100 (16 位补码) :

二进制: 11111111 10011100
十六进制: 0xFF9C

直接发给 PLC 会怎样? PC 是小端, 内存中低字节在前:

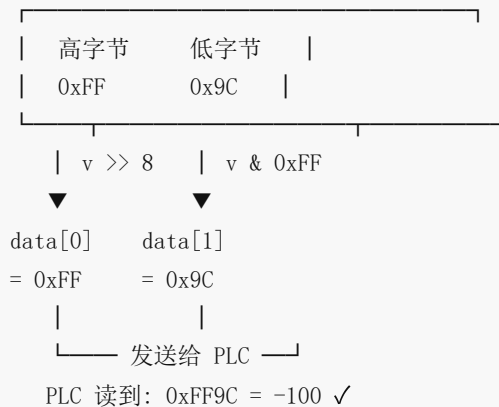
PC 内存: [9C] [FF] ← 低字节 0x9C 在前
PLC 收到: [9C] [FF] ← PLC 当作大端读
PLC 解读: 0x9CFF = 40191 (完全错误!)

正确做法——打包成大端:

```
QByteArray data(2, '\0');           // 2 字节缓冲区
data[0] = (char)(((quint16)v >> 8) & 0xFF); // 高字节 → 地址 0
data[1] = (char)((quint16)v & 0xFF);      // 低字节 → 地址 1
// 结果: [FF] [9C] → PLC 正确解读为 -100
```

图解:

值 v = -100 (0xFF9C)



FLOAT32 怎么打包?

float 不能直接位移, 先用 `memcpy` 提取底层二进制位:

```
float f = 3.14;
quint32 bits;
std::memcpy(&bits, &f, 4); // 把 float 的 4 字节“偷看”成整数
// bits = 0x4048F5C3

// 然后和 INT32 一样大端拆分:
data[0] = (bits >> 24) & 0xFF; // 0x40
data[1] = (bits >> 16) & 0xFF; // 0x48
data[2] = (bits >> 8) & 0xFF;  // 0xF5
```

```
data[3] = bits & 0xFF;           // 0xC3
// PLC 收到: 40 48 F5 C3 → 3.14 ✓
```

关键代码讲解

plc_worker.h 新增内容

```
public slots:
    // 新增: 写入数据。data 已经由调用方打包成大端字节序
    void writeArea(int areaType, int dbNumber, int start,
                   const QByteArray &data);

signals:
    // 新增: 写入完成通知
    void dataWritten(int areaType, int dbNumber, int start);
```

plc_worker.cpp 新增内容

```
void PlcWorker::writeArea(int areaType, int dbNumber, int start,
                           const QByteArray &data)
{
    switch (areaType) {
        case AREA_M:
            result = m_client->MBWrite(start, data.size(),
                                       (void *)data.constData());

            break;
        case AREA_DB:
            result = m_client->DBWrite(dbNumber, start, data.size(),
                                       (void *)data.constData());

            break;
        // I 区、Q 区同理...
    }
}
```

要点:

- `data.constData()` 返回 `const char*`, 即数据的首地址
- `snap7` 的写函数参数结构和读完全对称: `Write(DB号/起始, 大小, 数据指针)`
- `(void *)` 强制转换是因为 `snap7` 声明为 `void *` 参数

mainwindow.cpp 新增: `packValue()`

这是项目 4 最核心的新函数。根据用户写的文本和类型, 打包成 PLC 能理解的大端字节:

```

QByteArray MainWindow::packValue(int valueType, const QString &text) const
{
    bool ok = false;
    switch (valueType) {
    case 0: // BOOL —— 1 字节, 0 或 1
        return QByteArray(1, (char)(text.toInt(&ok) ? 1 : 0));
    case 1: // INT8 —— 1 字节
        return QByteArray(1, (char)text.toInt(&ok));
    case 3: // INT16 —— 2 字节, 大端排列
        // >> 8 提取高字节, & 0xFF 提取低字节
        break;
    case 5: // INT32 —— 4 字节, 大端排列
        // >> 24 / >> 16 / >> 8 / & 0xFF
        break;
    case 6: // FLOAT32 —— 4 字节, 先 memcpy 再大端排列
        break;
    }
    return QByteArray(); // 失败返回空
}

```

为什么用 `& 0xFF` ? 位移后可能有多余高位, `& 0xFF` 只取最低 8 位。

mainwindow.cpp 新增: 写后回读

```

void MainWindow::onWriteClicked()
{
    // 1. 打包数据
    QByteArray packed = packValue(valType, text);

    // 2. 发送写请求
    emit requestWrite(area, dbNum, start, packed);

    // 3. 立即读回验证
    emit requestRead(area, dbNum, start, packed.size());
}

```

写完立刻读同一个地址——这是一种简单可靠的验证方法。如果回读的值和你写的一致, 说明写入成功。

UI 布局

连接设置	
PLC IP: [192.168.0.1] [连接] [断开]	
● 已连接	

读取数据	
区域: [M ▼] 起始: [0] 字节数: [10]	
解析为: [INT16 ▼] [读取]	
原始 HEX: 00 0A FF 3C ...	
解析值: 10, -196, ...	
写入数据 (新增)	
区域: [M ▼] 起始: [0]	
类型: [INT16 ▼] 值: [100] [写入]	
写入成功 (已自动回读验证)	

编译步骤

```
# 1. 配置
cmake -B build -G "Ninja" \
    -DCMAKE_CXX_COMPILER=g++ \
    -DCMAKE_PREFIX_PATH=C:/Qt/6.11.0/mingw_64

# 2. 编译
cmake --build build

# 3. 部署 DLL
cp snap7.dll build/
export PATH="/c/Qt/6.11.0/mingw_64/bin:$PATH"
cd build
windeployqt Stage4_WriteData.exe
```

常见问题

Q: 写入后回读到的值不对

原因: 字节序搞反了 / 类型选错了 / 起始地址写错了。 **解决:** 先读取该地址看当前值 → 写入 → 再读 → 对比三次数值, 判断是打包问题还是地址问题。

Q: 写入报错

原因: 写入的地址不可写 (比如某些 I 区地址只读) / DB 块未取消优化访问。 **解决:**

- M 区和 DB 块通常可读写
- I 区是输入映像, 实际物理输入决定其值, 写入后会被硬件刷新覆盖
- Q 区可写, 但可能受程序逻辑影响

Q: 写入 BOOL 没效果

原因：BOOL 写入整字节，如果该字节其他位被 PLC 程序控制，会立刻被覆盖。**解决：**确认 PLC 程序中没有对该地址的写操作。S7-1200 程序中如果用了 `=`（赋值线圈），HMI/上位机写入后会被程序立即覆盖。

Q: 输入的值格式不正确

原因：比如 INT16 类型输入了 `3.14`（浮点数）或 `99999`（超出范围）。**解决：**检查 `packValue()` 函数的返回值，空表示打包失败，界面上会提示"格式不正确"。

项目 4 vs 之前项目的术语对比

概念	读取（项目3）	写入（项目4）
snap7 函数	MRead / DBRead	MWrite / DBWrite
字节序处理	解析：大端 → PC 数值	打包：PC 数值 → 大端
关键操作	(高 << 8) 低	>> 8 / >> 16 / & 0xFF
数据流向	PLC → QByteArray → 解析 → 显示	用户输入 → packValue → QByteArray → PLC